Laboratorio de Redes con *Network Namespaces*

Un recorrido paso a paso: desde dos ordenadores unidos por un cable hasta una red con Internet, DHCP, DNS, web, VPN, Tor y detección de ataques. Todo con `ip netns`, sin programas complicados.

ANTES DE EMPEZAR

¿Qué es esto y cómo funciona?

Este laboratorio monta redes de ordenadores **dentro de tu propio PC**, sin máquinas virtuales. Para ello usamos tres piezas del kernel de Linux:

> **Network namespace (netns)**: una "caja" aislada con su propia red (interfaces, IPs, rutas, firewall). Cada caja se comporta como un ordenador independiente. Lo creamos con `ip netns add`.

> **veth**: un cable de red virtual con dos extremos. Lo que entra por un extremo sale por el otro. Conecta dos cajas, o una caja con un switch.

> **bridge**: un switch virtual. Aprende qué equipo hay en cada puerto y reparte el tráfico. No necesita IP.

CÓMO SE USA

Cada fase es un script. Las fases **01 a 05** son independientes (cada una monta su red desde cero). A partir de la **06** se construye una sola red que va creciendo, así que esas se ejecutan en orden:

```
sh 01_pc_pc.sh          # fases 01-05: pruébalas sueltas
sh 06_internet.sh       # base de las siguientes
sh 07_dhcp.sh
sh 08_dns.sh
sh 09_web.sh
sh 10_vpn.sh
sh 11_iptables.sh
sh 12_tor.sh            # tarda ~2 min (espera a que Tor arranque)
sh 13_deteccion_ataques.sh
```

Todos los comandos llevan `sudo` porque crear redes requiere permisos de administrador. La limpieza total está comentada al final de las fases 12 y 13.

LAS 13 FASES

1. PC ↔ PC
2. PC — Switch — PC
3. Dos redes y un router
4. Cadena de 4 routers
5. Red de routers (respaldo)
6. Internet e IPs públicas
7. DHCP
8. DNS
9. Servidores web
10. VPN (WireGuard)
11. Firewall (iptables)
12. Red Tor
13. Detección de ataques

FASE 01 · 01_pc_pc.sh

PC ↔ PC

Conectar dos ordenadores directamente con un cable.

```
[pc1] 10.0.0.1 ═══════════════ 10.0.0.2 [pc2]
```

PASOS

1. Crear dos cajas: `ip netns add pc1` y `pc2`.
2. Crear un cable: `ip link add veth_pc1_pc2 type veth peer name veth_pc2_pc1`.
3. Meter cada extremo en su PC: `ip link set veth_pc1_pc2 netns pc1`.
4. Encender las interfaces (nacen apagadas), incluida `lo`.
5. Poner una IP a cada una, en la misma red `10.0.0.0/24`.

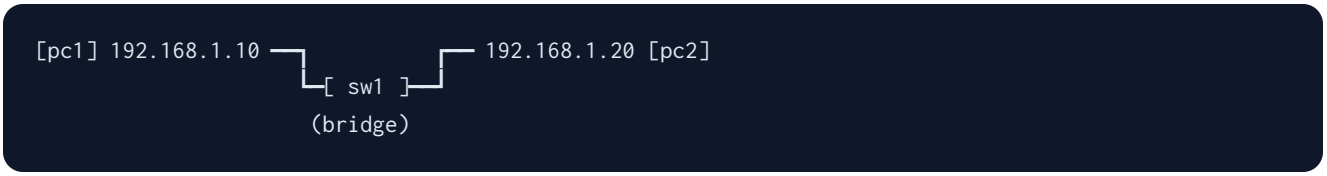
> Como están en la misma red, se ven directamente: no hace falta router.

Comprobación: `ip netns exec pc1 ping -c3 10.0.0.2` y `ip neigh` para ver la MAC de pc2 aprendida por ARP.

FASE 02 · 02_pc_sw_pc.sh

PC — Switch — PC

En vez de un cable directo, usar un switch para poder enchufar muchos PCs.



LO NUEVO

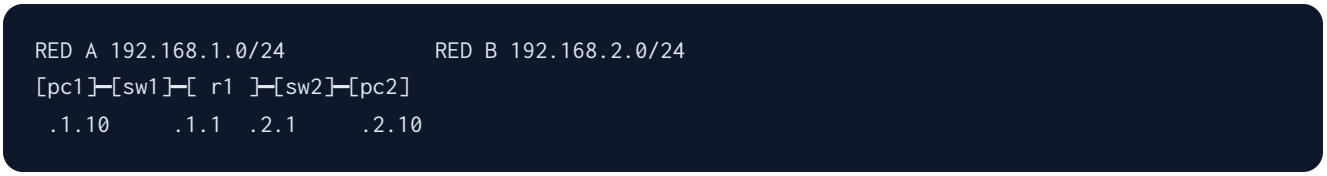
- Creamos un switch: `ip link add sw1 type bridge`.
- El extremo "PC" del cable va dentro del PC; el extremo "switch" se enchufa con `ip link set veth_sw1_pc1 master sw1`.
- El switch no lleva IP: solo reenvía tramas.

Comprobación: `ping` entre los PCs y `bridge fdb show br sw1` para ver la tabla de MACs que ha aprendido el switch.

FASE 03 · 03_pc_sw_r_sw_pc.sh

Dos redes unidas por un router

Comunicar dos redes distintas a través de un router.



> Un **router** es una caja con dos interfaces (una en cada red) y el reenvío activado: `sysctl -w net.ipv4.ip_forward=1`.

> La **puerta de enlace (gateway)** es a quién le manda un PC lo que no es de su red. Se pone con `ip route add default via 192.168.1.1`.

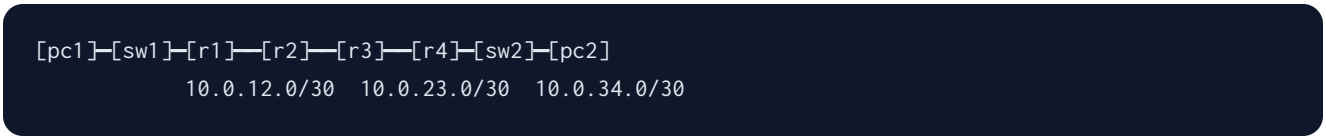
El router no necesita rutas: conoce las dos redes porque está conectado a ambas.

Comprobación: `ping` de pc1 a pc2 y `tracepath -n 192.168.2.10` para ver que el paquete pasa por r1.

FASE 04 · 04_cadena_routers.sh

Cadena de 4 routers

Varios routers en fila. Ahora cada uno NO conoce todas las redes: hay que enseñarle el camino con rutas estáticas.



LO NUEVO

- Enlaces punto a punto entre routers con redes `/30` (solo 2 IPs útiles).
- **Rutas estáticas**: "para llegar a la red X, manda el paquete al router Y".
- Hacen falta rutas de **ida** (hacia la red B) y de **vuelta** (hacia la red A) en cada router.

```
ip netns exec r1 ip route add 192.168.2.0/24 via 10.0.12.2
ip netns exec r2 ip route add 192.168.2.0/24 via 10.0.23.2
...
```

Comprobación: `tracepath` de pc1 a pc2 debe mostrar los 4 routers.

FASE 05 · 05_red_routers.sh

Red de routers con camino de respaldo

Los routers forman una malla con varios caminos. Si un enlace se cae, el tráfico va por otro lado.



> Con dos rutas a la misma red, se usa la de **métrica más baja**. La otra queda de reserva: `ip route add ... metric 100` / `metric 200`.

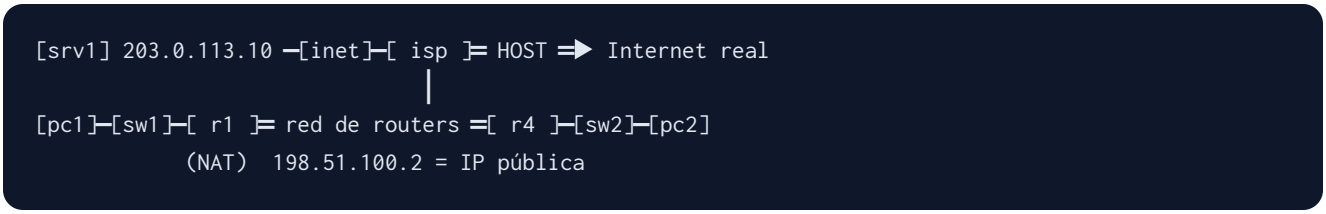
Con `ignore_routes_with_linkdown=1`, si un cable queda muerto el kernel deja de usar sus rutas y entra la de reserva.

Comprobación: el script apaga el enlace r1–r2 y comprueba que pc1 sigue llegando a pc2 por r3. Luego lo repara.

FASE 06 · 06_internet.sh

Internet e IPs públicas

Conectar la red a un "ISP" y, a través de él, a un Internet simulado y al Internet real del PC.



CONCEPTOS CLAVE

IP privada	192.168.x.x, 10.x.x.x — solo valen dentro de casa.
IP pública	Única en Internet (usamos los rangos de documentación 198.51.100.0/24 y 203.0.113.0/24).
NAT (MASQUERADE)	r1 cambia la IP privada de origen por su IP pública al salir. `iptables -t nat -A POSTROUTING -o veth_r1_isp -j MASQUERADE`
Ruta por defecto	"Lo que no sé dónde está, mándalo hacia Internet".

Comprobación: pc1 llega a srv1 (IP pública) y a `8.8.8.8` (Internet real). Desde Internet, srv1 *no* puede ver las IPs privadas: es lo que esperamos.

FASE 07 · 07_dhcp.sh

DHCP: IPs automáticas

Que los PCs nuevos reciban IP, gateway y DNS automáticamente, en vez de ponerlo a mano.

> El PC grita "¿alguien me da una IP?" (broadcast) y el servidor **DHCP** le contesta con IP + máscara + gateway + DNS, prestada por un tiempo (*lease*).

- Servidor: `dnsmasq` en r1 (red A) y r4 (red B), repartiendo el rango `.100-.150`.
- Cliente: `dhcpcd` en pc3 y pc4. Nacen **sin IP** a propósito.

```
dnsmasq --interface=veth_r1_sw1 \
  --dhcp-range=192.168.1.100,192.168.1.150,255.255.255.0,12h \
  --dhcp-option=option:router,192.168.1.1
```

Comprobación: `ip addr` en pc3 muestra la IP que recibió, y el archivo `.leases` registra a quién se la dio.

FASE 08 · 08_dns.sh

DNS: nombres en vez de IPs

Poder escribir `srv1.melvin.lab` en lugar de recordar la IP.

- Montamos un servidor DNS (`dnsmasq`) en Internet, en `203.0.113.53`.
- **Zona propia**: los nombres `*.melvin.lab` apuntados en un archivo tipo `/etc/hosts`.
- **Reenviador**: lo que no conoce (google.com…) se lo pregunta a `8.8.8.8`.
- A cada PC se le dice qué DNS usar creando `/etc/netns/<pc>/resolv.conf`.

Comprobación: `dig +short srv1.melvin.lab` devuelve la IP. Usamos `dig` porque respeta el `resolv.conf` del namespace.

FASE 09 · 09_web.sh

Servidores web

Montar una web pública en Internet y una intranet privada.

- Web pública en `203.0.113.80` (`web.melvin.lab`).
- Intranet en pc1 (`192.168.1.10`), accesible solo desde dentro.
- Servidor: `python3 -m http.server 80`. Registramos los nombres en el DNS y avisamos a dnsmasq con una señal `HUP`.

Comprobación: `curl http://203.0.113.80` funciona desde todas partes; la intranet se ve desde la red interna pero *no* desde Internet.

FASE 10 · 10_vpn.sh

VPN con WireGuard

Un usuario que está en Internet (su "casa") entra a las redes privadas como si estuviera dentro, por un túnel cifrado.

```
[casa] 203.0.113.100  ═══ TÚNEL CIFRADO (UDP 51820) ═══  [ r1 ] 198.51.100.2
  wg0: 10.10.0.2                                          wg0: 10.10.0.1
```

> **WireGuard**: cada extremo tiene una clave privada (secreta) y una pública (se la da al otro). La interfaz `wg0` cifra lo que sale y descifra lo que entra.

`allowed-ips` indica qué IPs van por el túnel: la del túnel (10.10.0.0/24) y las redes privadas (192.168.0.0/16).

Comprobación: antes de la VPN, casa no llega a pc1. Después, sí; y `wg show` muestra el último handshake.

FASE 11 · 11_iptables.sh

Firewall con iptables

Proteger la red: hasta ahora todo estaba permitido. Ponemos cortafuegos en r1 y en el servidor web.

INPUT	Tráfico dirigido al propio equipo.
FORWARD	Tráfico que el router reenvía.
Política DROP	"Todo prohibido salvo lo permitido": `iptables -P FORWARD DROP`.
ESTABLISHED,RELATED	Dejar pasar las respuestas de conexiones ya permitidas.
DNAT	Publicar un servicio interno: quien entra a `198.51.100.2:8080` llega a la intranet de pc1.

Comprobación: la red A sale a todo; la red B solo accede a la A por ping/web; desde Internet no se entra salvo por la VPN y el port forwarding. Las conexiones prohibidas dan *timeout* (DROP no responde).

FASE 12 · 12_tor.sh

Red Tor privada

Montar nuestra propia red Tor (no la real) y navegar de forma anónima desde pc1.

```
[ da ] autoridad + entrada
[ mid ] intermedio          circuito de 3 saltos cifrado
[ exit ] salida ──▶ [ web ]
pc1 = cliente Tor (SOCKS 127.0.0.1:9050)
```

> Un **circuito** usa 3 relays: entrada → intermedio → salida. Cada uno solo conoce al anterior y al siguiente, así que la web ve la IP del *exit*, nunca la de pc1.

> La **autoridad de directorio** lleva la lista de relays (el "consenso"). Usamos `TestingTorNetwork 1` para que arranque rápido y permita IPs privadas.

Comprobación: en el log de la web, la visita sin Tor llega desde 198.51.100.2 (r1); con Tor llega desde 203.0.113.203 (el exit). El origen real queda oculto.

FASE 13 · 13_deteccion_ataques.sh

Detección de ataques

Montar un servidor "víctima" y una máquina "atacante" en un segmento aislado, lanzar ataques clásicos contra nuestro propio servidor y aprender a verlos y bloquearlos.

```
[atacante] 10.13.13.20 ─[ sw13 ]─ 10.13.13.10 [victima]
                                    web: /login  /buscar
```

HERRAMIENTAS DE DETECCIÓN (TODO CON LO YA INSTALADO)

- **tcpdump**: graba el tráfico a un `.pcap` que se abre con Wireshark.
- **iptables -m string**: mira dentro del paquete y caza las palabras típicas de una inyección SQL.
- **iptables -m recent**: cuenta conexiones por IP y bloquea a quien abre demasiadas en poco tiempo.
- El **log del propio servidor**, que deja constancia de cada intento.

ATAQUE A · SQL INJECTION

Una web mal hecha construye la consulta pegando lo que escribe el usuario:

```
SELECT * FROM productos WHERE nombre = '<lo que escribe el usuario>'
```

Si el atacante escribe `' OR '1'='1` la condición siempre es verdadera; con `UNION SELECT` intentaría sacar otras tablas. Lo detectamos buscando esas palabras en el paquete:

```
iptables -A INPUT -p tcp --dport 80 \
  -m string --algo bm --icase --string "union select" \
  -j LOG --log-prefix "ALERTA-SQLI "
```

ATAQUE B · FUERZA BRUTA

El atacante prueba muchas contraseñas seguidas. Lo detectamos contando conexiones nuevas por IP:

```
iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW \
  -m recent --update --seconds 60 --hitcount 4 --name BF -j DROP
```

> **Usuarios y permisos en netns:** un namespace aísla la *red*, pero comparte los usuarios del sistema (no tiene su propio `/etc/passwd`). Por eso, en vez de una base de datos real, simulamos el login y la BD con un pequeño servidor en Python; se ve igual de bien y es más sencillo.

Informe final: el script muestra las alertas de iptables (`dmesg`), los contadores de reglas, las IPs en la lista `recent` , el log del servidor y el resumen del `.pcap` . Primero se ve el ataque *sin* defensas y luego *con* ellas, para comparar.

⚠ Todos los ataques se lanzan contra el servidor del propio laboratorio, en un segmento aislado y con fines educativos.